

第 7 集 · 约 18 分钟

# 桌面外壳：和 Windows 打交道

从命令行里的一个服务，到双击就能用的软件

配套阅读：书的第 7 章



这一集你会学到

1. `cfg`：编译时的 if，和 Python 的 `if os.name == "nt"` 差在哪
2. 调 Windows API：`unsafe`、UTF-16、以 0 结尾的字符串
3. `Drop` 和一个必踩的坑：`let _` 和 `let _lock` 完全不一样
4. 托盘为什么要占住主线程，tokio 运行时只好手动建

# 先猜一下

`let _ = 锁;` 和 `let _lock = 锁;` 只差四个字母。  
其中一行会让“只能开一个”悄悄失效，是哪一行？

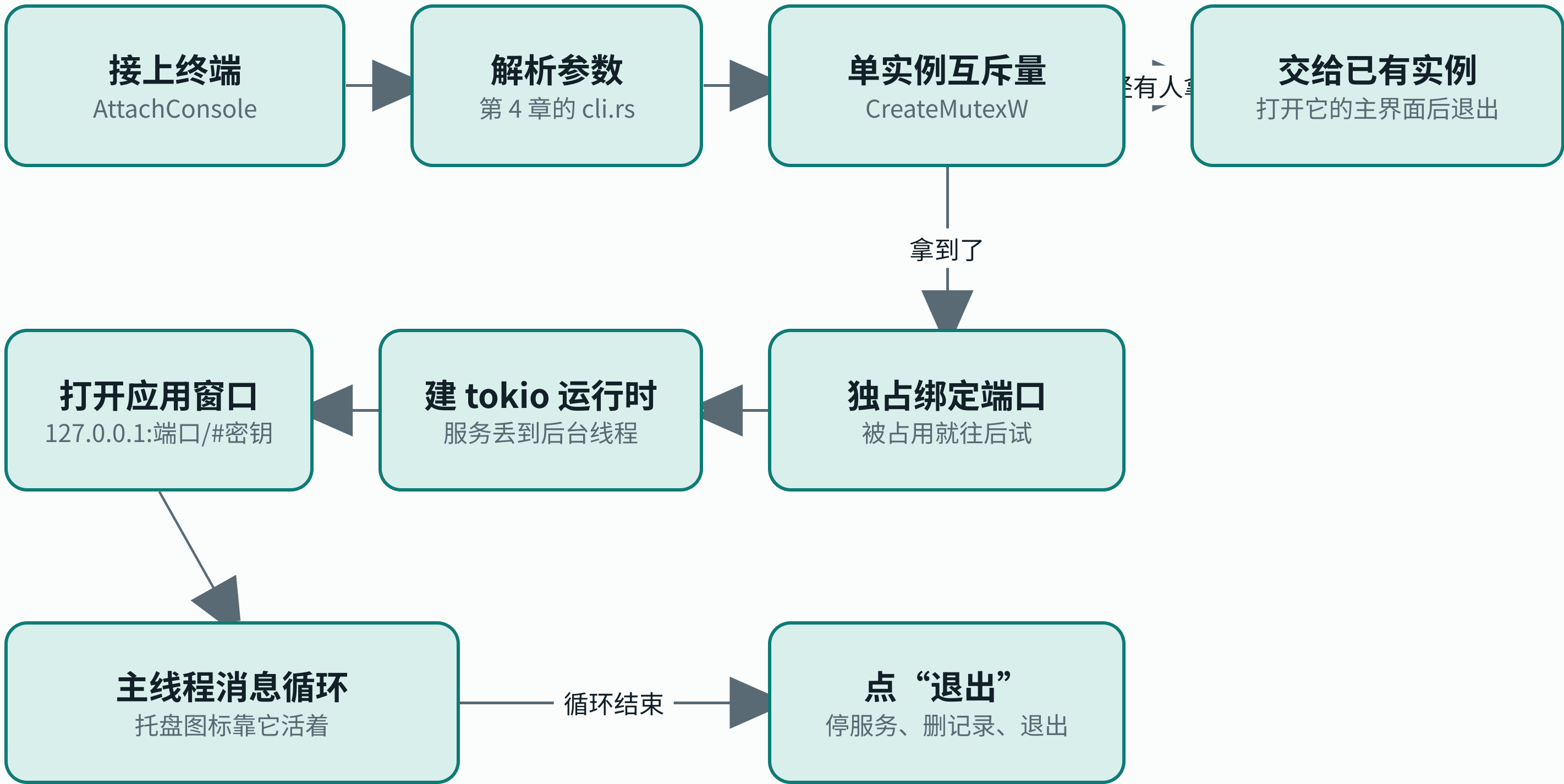
是 `let _ =`。而且编译器一个字都不会说。为什么？讲到 `Drop` 的时候揭晓。

# 目标：双击就是这个窗口



双击 exe 后自动打开的应用窗口

# 双击之后，main 做了什么



## cfg: 编译时的 if

```
1 // main.rs 第一行
2 #[cfg_attr(windows, windows_subsystem = "windows")]
3
4 // desktop/shell.rs
5 #[cfg(windows)]
6 pub fn shell_open(target: &str) { /* 调 ShellExecuteW */ }
7
8 #[cfg(not(windows))]
9 pub fn shell_open(target: &str) { /* 调 xdg-open */ }
```

# 动手：换个系统编译

目标平台：

在 Windows 上编译

在 Linux 上编译

## Rust：编译时选择

```
1  #[cfg(windows)]
2  pub fn shell_open(target: &str) { /* 调 ShellExecuteW */ }
3
4  #[cfg(not(windows))]
5  pub fn shell_open(target: &str) { /* 调 xdg-open */ }
```

划掉的那段**不会被编进程序**（只要求语法正确，类型检查都跳过）：在 Windows 上编译出来的程序里只有一个 shell\_open。

## Python：运行时判断

```
1  if os.name == "nt":
2      os.startfile(target)
3  else:
4      subprocess.run(["xdg-open", target])
```

两段代码都在程序里，跑到这一行才判断走哪段。

# 照着 ctypes 的习惯写

Windows PowerShell

```
PS D:\lanshare> cargo build
```

```
error[E0308]: arguments to this function are incorrect
```

```
note: expected `*const u16`, found `&str`
```

```
    = note: expected raw pointer `*const u16`
```

```
              found reference `&'static str`
```

```
PS D:\lanshare> cargo build
```

```
error[E0133]: call to unsafe function `MessageBoxW` is unsafe and requires unsafe block
```

# ctypes 对照 Rust：谁替你检查

Python v1 · desktop.py

```
1 def show_error(message):
2     """没有控制台时用弹窗告诉用户出了什么事。"""
3     if os.name == "nt":
4         try:
5             import ctypes
6             ctypes.windll.user32.MessageBoxW(None, message)
7             return
8         except (AttributeError, OSError):
9             pass
10    print(message, file=sys.stderr)
```

Rust v2 · shell.rs

```
1 pub fn show_error(message: &str) {
2     #[cfg(windows)]
3     {
4         use windows_sys::Win32::UI::WindowsAndMessaging::MessageBoxW;
5         let wide = |s: &str| s.encode_utf16().chain(std::iter::repeat(0)).collect();
6         let (text, title) = (wide(message), wide("局域网快件"));
7         // SAFETY: 两个字符串都以 0 结尾、调用期间有效；父窗口为 0
8         unsafe { MessageBoxW(std::ptr::null_mut(), text.as_ptr(), title.as_ptr(), 0) };
9     }
10    eprintln!("{message}");
11 }
```

# Drop：离开作用域自动清理

desktop/instance.rs、main.rs

```
1 pub struct InstanceLock { handle: HANDLE }
2
3 impl Drop for InstanceLock {
4     fn drop(&mut self) {
5         // SAFETY: handle 来自 CreateMutexW, 只在这里关闭一次。
6         unsafe { CloseHandle(self.handle) };
7     }
8 }
9
10 // main.rs 里这样持有它
11 let _lock = if args.single_instance { ... InstanceLock::acquire(MUTEX_NAME) ... };
```

# 动手：let \_ 和 let \_b

```
1 struct Lock(&'static str);    // 被丢弃时打印“释放 ...”
2 let _ = Lock("A");           // 下划线本身：值立刻被丢弃
3 let _b = Lock("B");           // 下划线开头的名字：活到作用域结束
4 println!("main 里的工作做完了");
5 }
```

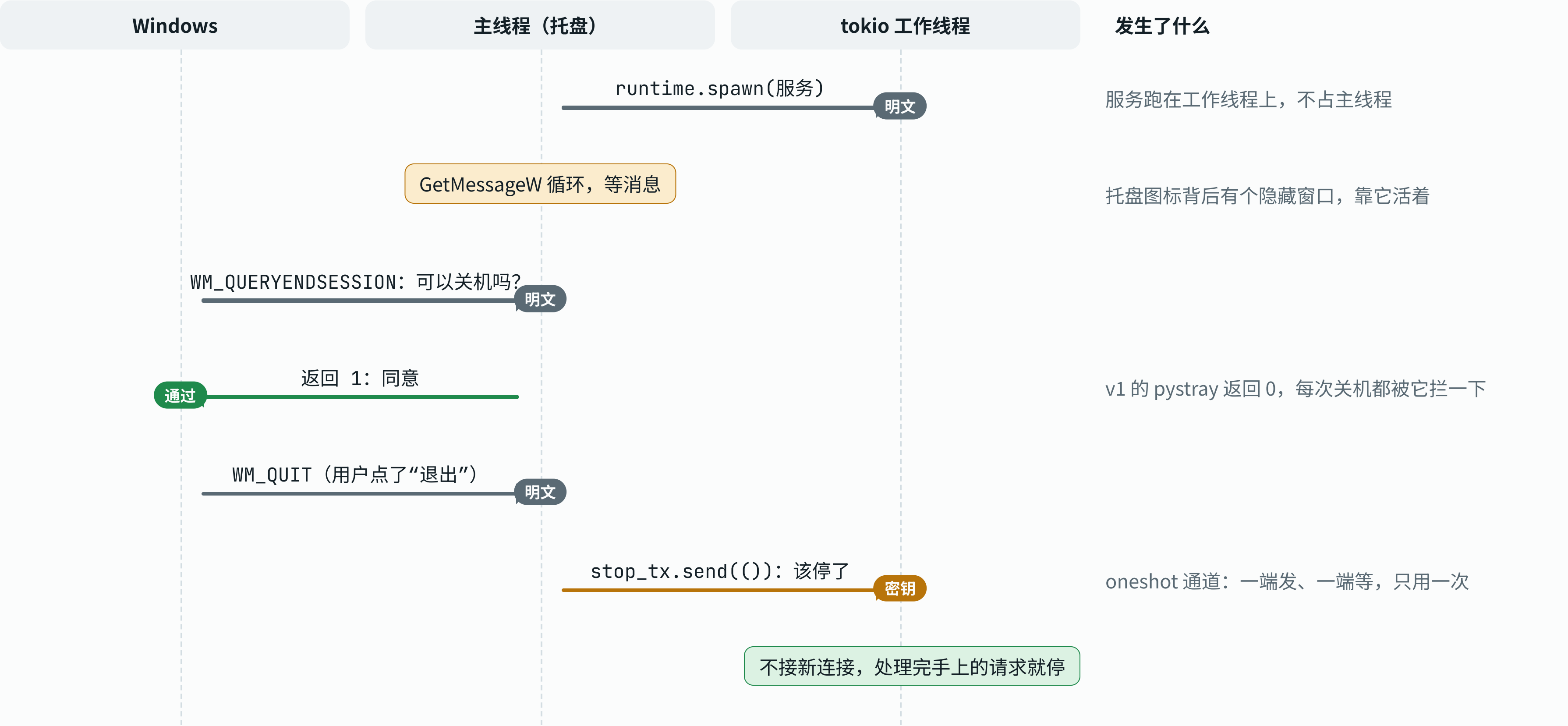
现在还活着的值  
(无)

## 控制台输出

释放 A  
main 里的工作做完了  
释放 B

到了右大括号，B 才被丢弃。这就是 Drop：离开作用域一定执行。

# 主线程留给托盘，服务交给 tokio



# 真实踩坑

好几次，程序本身没错，错的是“环境”和“旧数据”

**自动化测试里，--help 弹出一个对话框，测试卡着等人点“确定”**  
原来按 AttachConsole 成没成功判断有没有控制台；改成看标准输出句柄能不能用

**崩溃留下旧的实例记录，后来者去开一个早就关掉的端口**  
拿到互斥量后第一件事就删旧记录。我的测试脚本也犯过同样的错，报 ECONNREFUSED 127.0.0.1:51319

**测“Ctrl+C 能退出”，程序就是不退**  
从 Git Bash 启动的进程继承了“忽略 Ctrl+C”。先 SetConsoleCtrlHandler(NULL, FALSE)，立刻退出，退出码 0

**读状态文件之前先问一句：这个文件是这一次写的吗？**

选择题

## 下面这样写，会发生什么？

```
1 let p = wide(message).as_ptr();  
2 unsafe { MessageBoxW(std::ptr::null_mut(), p, title.as_ptr(), MB_ICONERROR) };
```

A 编译不过，借用检查会发现 Vec 已经被释放了

B 能编译（编译器会给一条警告），但 p 已经是悬空指针

C 没问题，Vec 会活到函数结束

D 运行时 Rust 会 panic 提醒你

选择题

下面这样写，会发生什么？

```
1 let p = wide(message).as_ptr();
2 unsafe { MessageBoxW(std::ptr::null_mut(), p, title.as_ptr(), MB_ICONERROR) };
```

A 编译不过，借用检查会发现 Vec 已经被释放了

借用检查管的是引用，不管裸指针。实测是能编译的，只多了一条警告。

B 能编译（编译器会给一条警告），但 p 已经是悬空指针

`wide(message)` 返回的 Vec 是个临时值，没绑定到变量，这条 let 语句一结束就被释放。新版编译器认得这种常见写法，会给 `dangling_pointers_from_temporaries` 警告，但只是警告。

C 没问题，Vec 会活到函数结束

绑定到变量上的值才活到作用域结束。没绑定的临时值，只活到这条语句结束。

D 运行时 Rust 会 panic 提醒你

Rust 不在运行时检查裸指针。编译时最多给条警告，写法绕一点连警告都没有，所以它要求你写 `unsafe`。

选择题

**LanShare 的 main 为什么不写 `#[tokio::main]`，而是手动建运行时？**

A `#[tokio::main]` 在 Windows 上用不了

B 托盘的消息循环要占着主线程，`#[tokio::main]` 也要用主线程驱动运行时

C 手动建的运行时跑得更快

D 为了让 Ctrl+C 能退出

选择题

# LanShare 的 main 为什么不写 `#[tokio::main]`，而是手动建运行时？

A `#[tokio::main]` 在 Windows 上用不了

能用。服务端本来就是 tokio 写的，在 Windows 上跑得好好的，问题不在平台。

B 托盘的消息循环要占着主线程，`#[tokio::main]` 也要用主线程驱动运行时

对。消息循环必须跑在创建托盘图标的线程上，两边冲突。所以手动建运行时，用 `runtime.spawn` 把服务放到工作线程。

C 手动建的运行时跑得更快

这里跟速度无关，手动建只是为了把主线程空出来。

D 为了让 Ctrl+C 能退出

Ctrl+C 退出只在 `--no-tray` 前台模式下用，靠的是 `tokio::signal::ctrl_c()`，跟运行时怎么建设关系。

想一想

**Windows 上，`0.0.0.0:8000` 能和别的程序的 `127.0.0.1:8000` 同时存在，发往 `127.0.0.1` 的请求会进别人的程序，而本机的应用窗口用的正是 `127.0.0.1`。所以 `net.rs` 绑定前设了 `SO_EXCLUSIVEADDRUSE`，还写了测试。测试一直是绿的，你怎么确定它真能抓住问题，而不是“怎么写都会过”？**

想一想

**Windows 上，`0.0.0.0:8000` 能和别的程序的 `127.0.0.1:8000` 同时存在，发往 `127.0.0.1` 的请求会进别人的程序，而本机的应用窗口用的正是 `127.0.0.1`。所以 `net.rs` 绑定前设了 `SO_EXCLUSIVEADDRUSE`，还写了测试。测试一直是绿的，你怎么确定它真能抓住问题，而不是“怎么写都会过”？**

故意把防线改坏再跑：把 `exclusive(&socket)?`；这一行注释掉，测试 `port_taken_on_a_specific_address_is_skipped` 实测变成 FAILED；恢复这一行，又通过了。去掉防护测试就红，才说明它真的守着这条防线。这叫变异测试，第 9 集会大量用到。

# 这一集我们学了

- ❌ `cfg` 在编译时选代码，没选中的连类型检查都不做；Python 的 `if os.name` 是运行时判断
- ❌ 调 Windows API：字符串转成以 0 结尾的 UTF-16，`unsafe` 块旁边写一行 SAFETY 注释
- ❌ `let _ =` 当场丢弃，`let _lock =` 活到作用域结束：锁和托盘图标都要用后者
- ❌ 托盘要占主线程，所以手动建 tokio 运行时，服务交给工作线程
- ❌ 测试绿了不算数，把防线改坏它会变红才算

下一集

下一集：构建与发布。exe 能小到多少？会不会因此变慢？一条命令从源码到成品